

EXPERIMENT – 21

Implementation of ECB, CBC, and CFB Modes of Operation with Padding in C

CODE:

```
#include <stdio.h>

#define MAX 100
#define BLOCK_SIZE 8

// Simple XOR encryption function
unsigned char encrypt(unsigned char data, unsigned char key)
{
    return data ^ key;
}

int main()
{
    unsigned char plaintext[MAX], key, iv;
    unsigned char ecb[MAX], cbc[MAX], cfb[MAX];
    unsigned char temp;
    int n, i;

    printf("Enter number of data blocks: ");
    scanf("%d", &n);

    printf("Enter %d plaintext blocks (0-255):\n", n);
    for (i = 0; i < n; i++)
        scanf("%hhu", &plaintext[i]);
```

```
printf("Enter key (0-255): ");
```

```
scanf("%hhu", &key);
```

```
printf("Enter IV (0-255): ");
```

```
scanf("%hhu", &iv);
```

```
// ECB MODE
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    ecb[i] = encrypt(plaintext[i], key);
```

```
}
```

```
// CBC MODE
```

```
temp = iv;
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    cbc[i] = encrypt(plaintext[i] ^ temp, key);
```

```
    temp = cbc[i];
```

```
}
```

```
// CFB MODE
```

```
temp = iv;
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    cfb[i] = plaintext[i] ^ encrypt(temp, key);
```

```
    temp = cfb[i];
```

```
}
```

```

// Display results

printf("\nECB Encrypted Blocks:\n");

for (i = 0; i < n; i++)

    printf("%d ", ecb[i]);


printf("\n\nCBC Encrypted Blocks:\n");

for (i = 0; i < n; i++)

    printf("%d ", cbc[i]);


printf("\n\nCFB Encrypted Blocks:\n");

for (i = 0; i < n; i++)

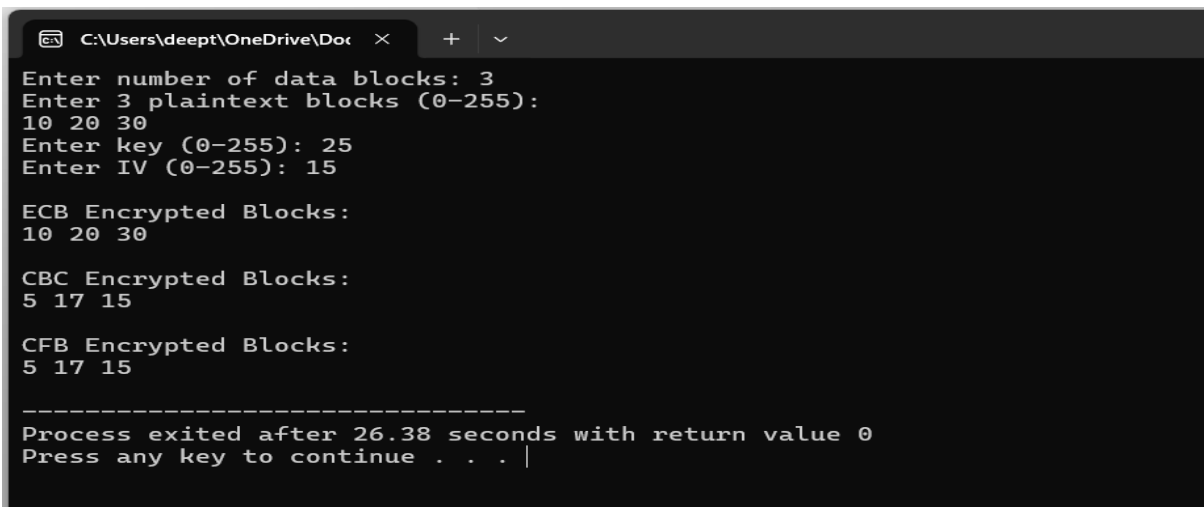
    printf("%d ", cfb[i]);


printf("\n");


return 0;
}

```

OUTPUT:



```

C:\Users\deepth\OneDrive\Doc
Enter number of data blocks: 3
Enter 3 plaintext blocks (0-255):
10 20 30
Enter key (0-255): 25
Enter IV (0-255): 15

ECB Encrypted Blocks:
10 20 30

CBC Encrypted Blocks:
5 17 15

CFB Encrypted Blocks:
5 17 15

-----
Process exited after 26.38 seconds with return value 0
Press any key to continue . . . |

```